



WEB DEVELOPMENT

Lesson 8

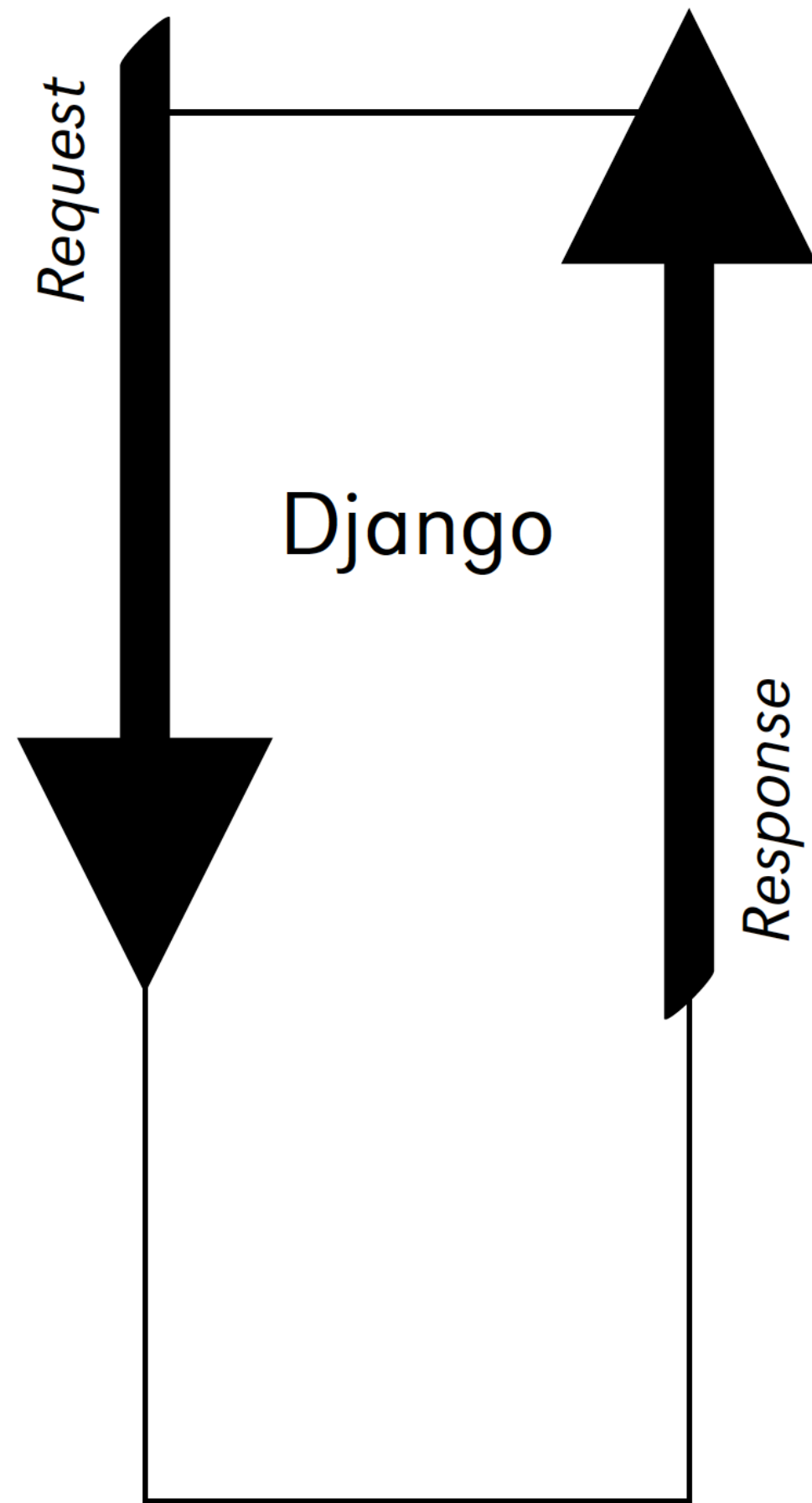
What is Django?

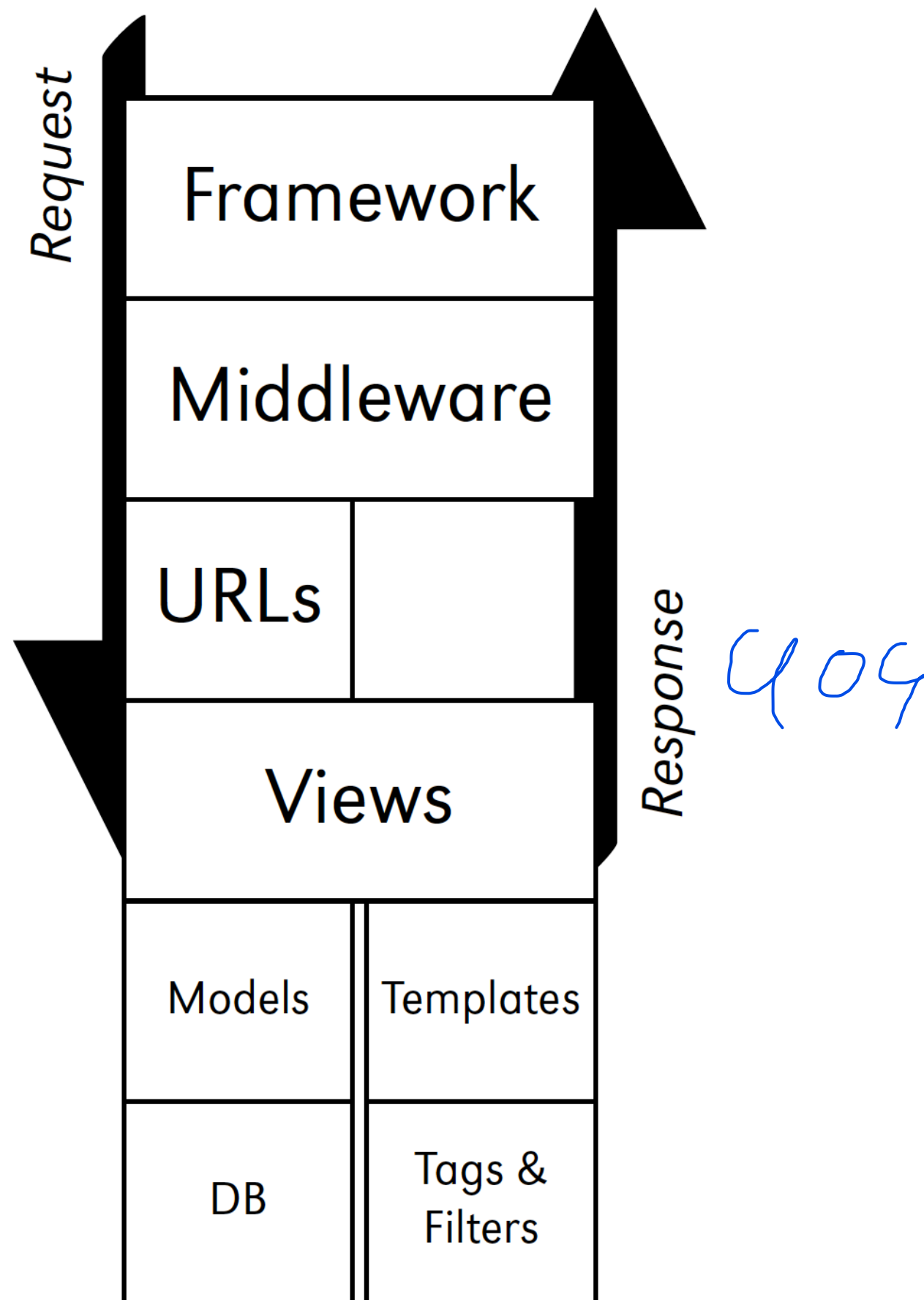
What is Django?

- High-level framework for rapid web development
- Complete stack of tools
- Data modelled with Python classes
- Production-ready data admin interface, generated dynamically
- Elegant system for mapping URLs to Python code
- Generic views' to handle common requests

Django Components

- Think MTV instead of MVC
- Models - Django ORM
- Templates - Django Template Engine
HTML
- Views - Python function, Request in Response out
исполняет запрос
- URL Patterns - Regular expression based





Defining Requirements

- requirements.txt

Create requirements.txt for current env

\$ **pip freeze > requirements.txt**

Install all modules from requirements.txt file recursive

\$ **pip install -r requirements.txt**

Starting a Project

Mac/Linux/etc...

\$ pip install django

\$ django-admin startproject demo

\$ cd demo

\$ python manage.py migrate

\$ python manage.py runserver

Windows

> pip install django

> python Scripts/django-admin.py startproject demo

> cd demo

> python manage.py migrate

> python manage.py runserver

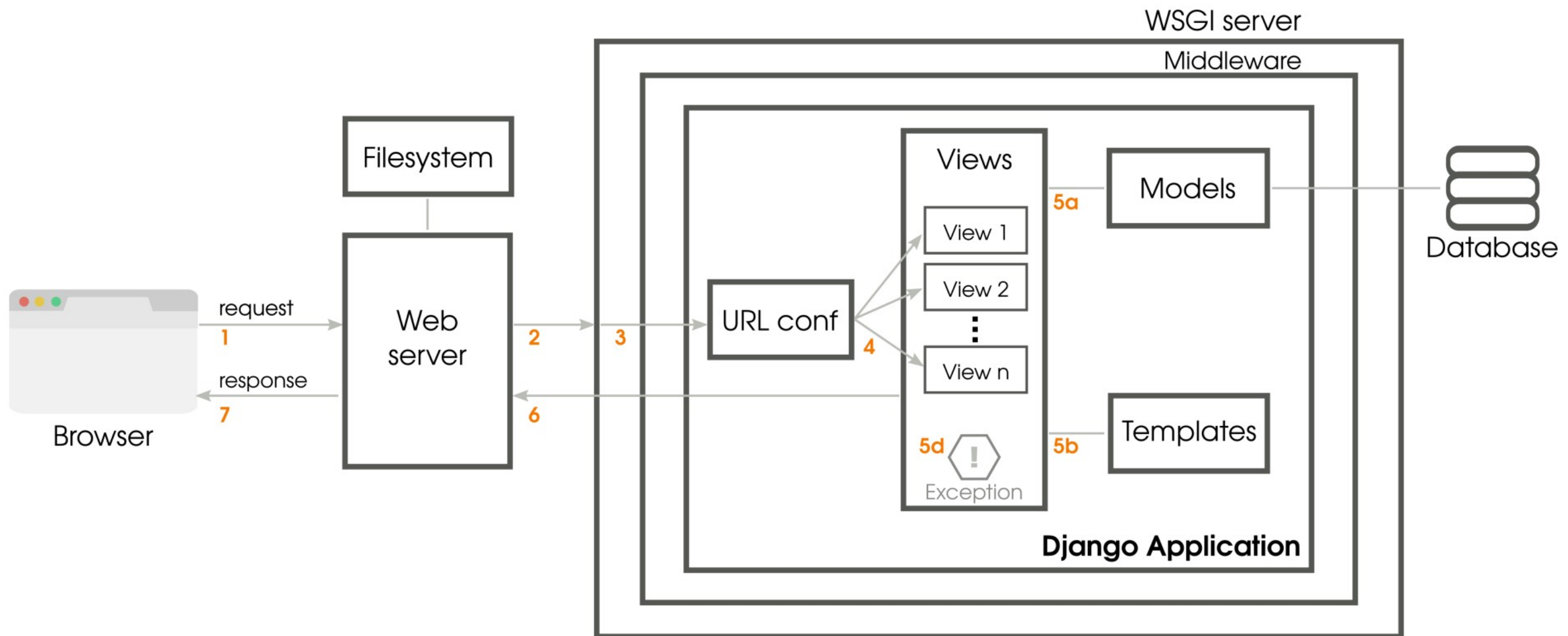
URLs

- Map URLs in requests to code that can be executed
- Regular expressions!
- Subsections of your site can have their own `urls.py` modules

Views

- Code that handles requests
- Other frameworks often call these “controllers”
- Basically a function that:
 - gets a request passed to it
 - returns text or a response

How Does Django Works?



Regular Expressions in URL

. — (dot) Any character

\d — Any digit

[A-Z] — Any character A-Z (uppercase)

[a-z] — Any character, a-z (lowercase)

[A-Za-z] — Any character, a-z (case insensitive)

+ — One or more of the previous expression

[^/]+ — All characters except forward slash

? — Zero or more of the previous expression

{1,3} — Between one and three (inclusive) of the previous expression

How Django Processes a Request

- `python manage.py runserver`
 - `settings.py`
 - `ROOT_URLCONF`
- `/time/`
 - look through all urls patterns and compare
 - `HttpRequest` object as first parameter
 - view function responsible to return `HttpResponse`

404 Errors

Word About Pretty URLs

```
re_path(r'^time/plus/\d+/$', hours_ahead)
```

- /time/plus/1/
- /time/plus/2/
- /time/plus/3/
- /time/plus/1000000/

Word About Pretty URLs

```
re_path(r'^time/plus/\d{1,2}/$', hours_ahead)
```

- /time/plus/**1**/
- /time/plus/**2**/
- /time/plus/**3**/
- /time/plus/1000000/

Passing that data to the view function

parentheses around the data

```
re_path(r'^time/plus/(\d{1,2})/$', hours_ahead)
```

Coding Order

1. views —> urls

2. urls —> views

Django new App

The Django Template System

template might be as HTML file

render function

MTV

- **M (Model)** — data access layer
- **T (Template)** — presentation layer
- **V (View)** — controller layer
- ***S (service layer)*** — *business logic layer*

Configuring the Database

Configuring the Database

`settings.py`

- `DATABASE_ENGINE = ''`
- `DATABASE_NAME = ''`
- `DATABASE_USER = ''`
- `DATABASE_PASSWORD = ''`
- `DATABASE_HOST = ''`
- `DATABASE_PORT = ''`

Configuring the Database

`settings.py`

- `DATABASE_ENGINE = ''`

Settings	Database	Required Adapter
<code>postgresql</code>	PostgreSQL	<code>psycopg</code> version 1.x
<code>postgresql_psycopg2</code>	PostgreSQL	<code>psycopg</code> version 2.x,
<code>mysql</code>	MySQL	<code>MySQLdb</code>
<code>sqlite3</code>	SQLite	<code>pysqlite</code>
<code>ado_mssql</code>	Microsoft SQL Server	<code>adodbapi</code> version 2.0.1+
<code>oracle</code>	Oracle	<code>cx_Oracle</code> , <code>oracledb</code>

Defining Models in Python

Python model  SQL **CREATE TABLE**

Create new migration file if necessary

```
>>> python manage.py makemigrations
```

Execute created migration file

(change database schemas)

```
>>> python manage.py migrate
```

Inserting and Updating Data

- Create instance of model
 - `save()` // INSERT INTO "table_name" ...
- Find object and update class fields
 - `save()` // UPDATE "table_name" SET ...

Selecting Objects

```
>>> products = Product.objects.all()
```

```
SELECT * FROM products;
```

Filtering Data

```
>>> p = Product.objects.filter(name="Test name")
```

```
SELECT * FROM products  
WHERE name='Test name';
```


Filtering Data

- `contains / icontains`
- `startswith / endswith`
- `istartswith / iendswith`
- `exact / iexact`
- `in`
- `isnull`
- `gt / gte`
- `lt / lte`

```
>>> p = Product.objects.filter(name__contains="Test")
```


Retrieving Single Objects

```
>>> p = Product.objects.get(id=3)
```

```
SELECT * FROM product WHERE id=3;
```

Ordering Data

```
>>> p = Product.objects.order_by("name")
```

```
SELECT * FROM products ORDER BY name;
```

Ordering Data

```
>>> p = Product.objects.order_by("name", "address")
```

```
>>> p = Product.objects.order_by("-name")
```

Meta class

```
class Product(models.Model):  
    ...  
    class Meta:  
        ordering = ["name"]
```

Chaining Lookups

```
>>> p = Product.objects.filter(name="Laptop").order_by("-price")
```

Deleting Objects

```
>>> p = Product.objects.get(id=3)
```

```
>>> p.delete()
```

```
>>> products = Product.objects.all()
```

```
>>> products.delete()
```

Questions?